

Spécification du sous-projet autolisp-hash-table

OpenAI

Objet

Le sous-projet **autolisp-hash-table** vise à fournir une implantation de **tables de hachage** pour AutoLISP, inspirée de Common Lisp, avec de bonnes performances pratiques sur les charges courantes.

Les objectifs principaux sont :

- une API proche de **make-hash-table**, **gethash**, **remhash**, **clrhash**, **maphash** et **sxhash** ;
- une stratégie de hachage de bonne qualité sur **tous les types AutoLISP** ;
- une implantation interne simple, robuste et benchmarkable ;
- une base réutilisable pour d'autres structures, notamment des vecteurs redimensionnables.

La v1 privilégie une **table ouverte** avec **sondage linéaire circulaire** :

- les entrées sont stockées dans un vecteur de slots ;
- en cas de collision, on cherche les cases suivantes jusqu'à trouver : une case vide, la clé, ou l'épuisement de la table ;
- lorsque la table devient trop pleine, on réalloue un vecteur plus grand et on réinsère les entrées existantes.

Principes de conception

Inspiration Common Lisp

L'API et la terminologie s'inspirent de Common Lisp :

- **make-hash-table** ;
- **gethash** ;
- **remhash** ;
- **clrhash** ;
- **hash-table-count** ;
- **maphash** ;
- **sxhash**.

L'implantation n'a pas besoin de reproduire toute la sémantique CL dès la v1, mais elle doit garder les mêmes idées directrices :

- distinction entre **test d'égalité** et **hachage** ;
- taille logique séparée de la capacité ;
- facteur de charge configurable ;
- croissance automatique ;
- réutilisation d'un **sxhash** générique.

Représentation externe

Une table de hachage est représentée par un **symbole**, comme les autres sous-projets AutoLISP du dépôt.

Les propriétés minimales attendues sont :

ah-kind marqueur de type, valeur **ah-hash-table** ;
ah-test prédicat d'égalité sélectionné ;
ah-count nombre d'entrées actives ;
ah-size capacité du vecteur de slots ;
ah-threshold seuil de charge avant croissance ;
ah-rehash-size politique de croissance ;
ah-rehash-threshold facteur de charge cible ;
ah-slots vecteur interne des slots ;
ah-deleted-marker marqueur interne des tombstones ;
ah-empty-marker marqueur interne des cases vides.

Tous les symboles publics et internes doivent être préfixés par **ah-** ou **ah--**.

Représentation interne

La table repose sur un **vecteur de slots** indexé par :

$\text{index} = \text{sxhash}(\text{key}) \bmod \text{capacity}$

Chaque slot peut être dans l'un des états suivants :

- **vide** ;
- **supprimé** (tombstone) ;
- **occupé** par une paire **key/value**.

Le sondage est **linéaire et circulaire** :

1. calcul de l'indice initial ;
2. test du slot courant ;
3. si collision, passage à l'indice suivant ;
4. retour à 0 après le dernier slot ;
5. arrêt après avoir exploré au plus **capacity** slots.

Cette stratégie est volontairement simple :

- elle évite les listes de collisions ;
- elle exploite bien le cache lorsqu'un vrai vecteur rapide est disponible ;
- elle reste correcte même si le pire cas est $O(n)$.

Politique de charge

La table ne doit pas approcher le remplissage complet.

La politique recommandée en v1 est :

- capacité choisie à partir d'un **nombre premier** ;
- croissance avant saturation ;
- seuil par défaut à **0.5** ;
- réallocation dès que `count > floor(size * threshold)`.

Le but est de conserver beaucoup de cases vides pour que le sondage linéaire reste rapide en pratique.

Objectifs de complexité

Cas moyen visé

gethash $O(1)$ amorti ;
insertion $O(1)$ amorti ;
suppression $O(1)$ amorti ;
lecture du nombre d'entrées $O(1)$;
vidage de la table $O(n)$;
réallocation $O(n)$, amortie sur les insertions.

Pire cas assumé

Avec le sondage linéaire, le pire cas reste :

recherche $O(n)$;
insertion $O(n)$;
suppression $O(n)$.

Ce pire cas est accepté, à condition que :

- la fonction de hachage soit de bonne qualité ;
- le seuil de charge reste bas ;
- la croissance soit correctement gérée.

État des benchmarks

Un banc d'essai est présent dans `tests/hash-table-benchmarks.lsp`.

État observé au 14 mars 2026 :

- la suite fonctionnelle passe ;
- l'exécution benchmark sous BricsCAD fournit maintenant un tableau exploitable via le runner dédié `tests/run-benchmarks.lsp` ;
- le wrapper `autolisp` recommande, sur macOS + BricsCAD, l'espace de travail `2D Drafting` plutôt que `2D Drafting (Modern)`.

Dernières mesures relevées :

opération	taille	itérations	ms	note
ah-puthash	11	11	2	count=11
ah-gethash	11	11	1	sum=55
ah-remhash	11	11	1	count=0
ah-puthash	23	23	4	count=23
ah-gethash	23	23	1	sum=253
ah-remhash	23	23	2	count=0
ah-puthash	47	47	10	count=47
ah-gethash	47	47	5	sum=1081
ah-remhash	47	47	6	count=0
ah-puthash	97	97	65	count=97
ah-gethash	97	97	46	sum=4656
ah-remhash	97	97	47	count=0

Ces chiffres restent indicatifs. La présente spécification ne fixe toujours aucun objectif normatif de temps d'exécution pour `ah-puthash`, `ah-gethash` ou `ah-remhash`.

API retenue en v1

Création et introspection

- `ah-make-hash-table` ;
- `ah-hash-table-p` ;
- `ah-hash-table-count` ;
- `ah-hash-table-size` ;
- `ah-hash-table-test` ;
- `ah-hash-table-rehash-threshold`.

Accès et mutation

- `ah-gethash` ;
- `ah-puthash` ou `ah-set-gethash` selon la convention retenue ;
- `ah-remhash` ;
- `ah-clrhash`.

Itération

- `ah-maphash`.

Hachage

- `ah-sxhash`.

La documentation devra toujours faire le lien avec les noms Common Lisp correspondants.

Contrat fonctionnel détaillé

(ah-make-hash-table &key test size rehash-size rehash-threshold)

Contraintes :

- `test` désigne une relation d'égalité supportée ;
- `size` est un indice de capacité initiale, pas un compte maximal garanti ;
- `rehash-threshold` est un réel strictement compris entre 0 et 1 ;
- `rehash-size` décrit la croissance.

Sémantique recommandée :

- la capacité réelle est le **plus petit nombre premier $\geq$ size minimal** ;
- si `size` n'est pas fourni, une petite capacité première par défaut est choisie ;
- la table créée est vide ;
- tous les slots internes sont initialisés à un marqueur `empty`.

(ah-gethash table key [default])

Retourne :

- la valeur associée à `key` si elle existe ;
- sinon `default` ou `nil`.

Convention retenue pour la v1 :

- `ah-gethash` retourne uniquement la valeur principale ;
- l'information secondaire de présence est stockée dans la variable globale `ah-*gethash-found-p*` ;
- après chaque appel, cette variable vaut T si la clé était présente, sinon `nil`.

Cette convention est provisoire et pourra être remplacée plus tard par de véritables valeurs multiples.

(ah-puthash table key value)

Effets :

- remplace la valeur si la clé existe déjà ;
- sinon insère une nouvelle entrée ;
- réutilise un slot **deleted** si disponible sur le chemin de sondage ;
- déclenche un rehash si le seuil de charge est atteint.

(ah-remhash table key)

Effets :

- si la clé existe, le slot devient **deleted** ;
- le compteur diminue ;
- la recherche future doit continuer à travers ce slot tombstone ;
- la fonction retourne uniquement la valeur supprimée ;
- l'information secondaire de succès est stockée dans la variable globale **ah-*remhash-found-p*** ;
- après chaque appel, cette variable vaut T si la clé était présente, sinon **nil**.

(ah-clrhash table)

Effets :

- remet le compteur à 0 ;
- réinitialise tous les slots à **empty** ;
- conserve la capacité courante.

(ah-maphash function table)

Parcourt toutes les entrées actives.

Contraintes v1 :

- l'ordre de parcours n'est pas spécifié ;
- la mutation structurelle pendant l'itération est soit interdite, soit explicitement laissée non spécifiée.

Fonction de hachage sxhash

Exigence générale

ah-sxhash est la pièce essentielle du projet.

Elle doit :

- être définie pour tous les types AutoLISP utiles ;
- être stable pendant une session ;

- produire un entier non négatif ;
- distribuer correctement les valeurs usuelles ;
- rester suffisamment rapide pour être appelée sur chaque accès.

La v1 retiendra un **sxhash explicite et documenté**, plutôt qu'un assemblage ad hoc de conversions implicites.

Types à couvrir

La v1 doit définir **ah-sxhash** pour au moins :

- **nil** ;
- entiers ;
- réels ;
- chaînes ;
- symboles ;
- listes ;
- dotted pairs ;
- objets VLA ;
- enames / entités CAD.

Pour les entités et objets CAD, on peut utiliser :

- le **handle** quand il est disponible ;
- sinon une représentation stable documentée par l'implémentation.

Normalisation par type

Avant mélange, la donnée à hacher doit être ramenée à une forme canonique interne.

La normalisation recommandée est :

- nil** tag dédié constant ;
- entier** valeur entière normalisée ;
- réel** représentation stable séparée de l'entier, même si la valeur numérique est proche ;
- chaîne** parcours caractère par caractère ;
- symbole** nom imprimé stable ;
- dotted pair** / **liste** hachage structurel récursif ;
- ename** / **objet CAD** type observable + handle canonique quand disponible ;
- objet inconnu** représentation imprimée documentée, en dernier recours.

Le **tag de type** doit participer au hash, pour éviter des collisions triviales entre valeurs de types différents.

Exemple d'intention :

- l'entier 12 ;

- la chaîne "12" ;
- le symbole |12| ;
- la liste '(1 2)

ne doivent pas être traités comme des entrées interchangeables.

Contraintes de cohérence

Pour un test d'égalité donné, il faut respecter :

`test(x, y) => compatible-hash(x) = compatible-hash(y)`

En pratique :

- avec `eq`, le hachage doit refléter une **représentation observable stable** ;
- avec `equal`, les objets structurellement égaux doivent avoir le même hash ;
- si plusieurs tests sont supportés, la `v1` retient un `hash function selector` dépendant du test.

Décision de conception :

- `ah-sxhash` désigne la famille de fonctions de hachage ;
- l'implantation concrète choisit `ah--sxhash-eq` ou `ah--sxhash-equal` suivant le test de la table ;
- toute table stocke donc implicitement **sa** politique de hachage.

Cette option est retenue car elle évite de surcontraindre `eq` pour satisfaire `equal`.

Politique par test

Test `eq`

AutoLISP ne donne pas accès de façon générale à l'identité mémoire ou à l'adresse des objets. La `v1` ne doit donc **pas** supposer l'existence d'un hash par adresse.

Le hash en mode `eq` s'appuie sur la **représentation observable la plus stable** disponible pour le type considéré.

Recommandations :

- symbole** nom du symbole, combiné avec un tag de type ;
- entité CAD** type d'entité + handle ;
- objet VLA** handle si disponible, sinon représentation stable documentée ;
- nombres immédiats** valeur normalisée.

Le but du mode `eq` est de privilégier la rapidité.

Test equal

Le hash doit être **structurel**.

Recommandations :

- chaînes égales -> même hash ;
- listes égales récursivement -> même hash ;
- dotted pairs égales -> même hash ;
- symboles portant le même nom logique selon la sémantique retenue -> même hash ;
- handles identiques -> même hash pour les objets CAD.

Le but du mode **equal** est de privilégier la correction sémantique.

Qualité du mélange

Le projet doit retenir une fonction de mélange de bonne qualité.

Exigences :

- bon effet d'avalanche ;
- faible biais sur les petites variations ;
- coût raisonnable en AutoLISP ;
- opération possible avec les entiers disponibles.

La v1 privilégie une fonction de mélange **simple, robuste et portable** plutôt qu'une fonction très sophistiquée dépendante de primitives bitwise avancées.

Les candidats typiques sont :

- FNV-1a ;
- Jenkins one-at-a-time ;
- Murmur-like mix simplifié ;
- combinaison multiplicative + xor + rotations simulées.

Décision provisoire de la spécification :

- la v1 prendra comme base **Jenkins one-at-a-time** ou une variante très proche ;
- si les opérations bitwise disponibles s'avèrent trop pauvres, on utilisera une adaptation multiplicative documentée qui préserve le même esprit de mélange ;
- toute déviation par rapport à Jenkins devra être justifiée dans la documentation du code.

Justification :

- meilleur compromis entre qualité et simplicité ;
- bon comportement sur des flux de caractères ;
- applicable aux chaînes, symboles, handles et sérialisations légères ;
- plus réaliste en AutoLISP qu'un Murmur complet.

Domaine numérique du hash

Le hash final doit être un entier non négatif borné par une plage sûre pour les entiers AutoLISP.

La v1 retiendra un masquage ou une réduction stable vers une plage documentée, par exemple :

- entier signé positif sur 29 à 31 bits ;
- toujours ensuite réduit modulo la capacité de la table.

L'important est moins la taille absolue que :

- la stabilité ;
- la bonne dispersion ;
- l'absence de valeurs négatives surprises.

Hachage des chaînes

Les chaînes sont hachées caractère par caractère.

Règles :

- parcours séquentiel de gauche à droite ;
- incorporation du code caractère ;
- mélange à chaque étape ;
- distinction explicite entre chaîne vide et autres valeurs.

La question de la casse dépend du test d'égalité retenu :

- en v1, sauf mention contraire, les chaînes sont hachées **telles quelles** ;
- un éventuel mode case-insensitive devra définir son propre hash compatible.

Hachage des symboles

Comme l'identité mémoire n'est pas accessible de façon portable, la v1 fixe une politique simple :

- un symbole est haché à partir de son **nom imprimé stable** ;
- le hash incorpore un **tag de type symbole** ;
- en pratique, pour **eq** comme pour **equal**, le nom du symbole sert donc de base canonique.

Cela est valable tant que le projet suppose qu'un même symbole est identifié par son nom interné observable.

Hachage des nombres

Les nombres doivent éviter les collisions triviales entre entiers et réels.

Règles recommandées :

- tag de type distinct pour entier et réel ;
- pour l'entier, mélange de la valeur brute ;
- pour le réel, utilisation d'une représentation stable : chaîne normalisée, mantisse/exposant, ou autre encodage documenté.

La représentation choisie devra garantir qu'un même réel haché deux fois dans la même session donne la même valeur.

Hachage structurel des listes

Pour les listes, on veut un hachage récursif, par exemple :

- hachage du `car` ;
- combinaison avec le hachage du `cdr` ;
- ajout d'un marqueur de structure pour distinguer listes et atomes.

Il faut éviter :

- une simple somme ;
- une simple concaténation naïve ;
- les collisions triviales sur permutations.

Règles supplémentaires :

- incorporer un tag `cons` à chaque nœud ;
- distinguer explicitement la fin de liste `nil` ;
- traiter correctement les dotted pairs, donc ne pas supposer qu'un `cdr` est toujours une liste.

Hachage des objets CAD

Pour les `ename` et objets VLA, la recommandation de la v1 est claire :

- utiliser le **handle** comme identité canonique lorsqu'il existe ;
- incorporer aussi le **type observable** de l'objet ou de l'entité.

Conséquences :

- deux références vers le même objet CAD doivent produire le même hash ;
- deux objets de types différents mais de handles textuellement identiques ne doivent pas être forcés dans le même hash avant mélange ;
- la qualité du hash dépend alors de la qualité du mélange appliqué au couple **type + handle** ;
- la récupération du handle doit rester hors du chemin critique des réinsertions si possible, en évitant les conversions répétées inutiles.

Si un objet n'expose pas de handle stable, l'implémentation devra documenter le fallback retenu.

Heuristiques retenues par type

La v1 retient les heuristiques concrètes suivantes :

- nil** constante taggée ;
- entier** tag entier + valeur ;
- réel** tag réel + représentation numérique stable ;
- chaîne** tag chaîne + contenu complet ;
- symbole** tag symbole + nom du symbole ;
- liste** tag cons + combinaison récursive du **car** et du **cdr** ;
- dotted pair** même règle que liste, sans hypothèse sur le **cdr** ;
- entité CAD** tag entité + type DXF + handle ;
- objet VLA** tag objet + nom de classe ou type observable + handle si disponible ;
- objet inconnu** tag objet inconnu + représentation imprimée stable.

Le principe général est :

- **toujours combiner le type et une représentation observable stable** ;
- **ne jamais dépendre d'une adresse mémoire non accessible.**

Invariants à tester

Les tests de **sxhash** devront vérifier au minimum :

- même objet, même hash ;
- objets égaux selon le test choisi, même hash ;
- objets de types différents, hashes souvent différents ;
- absence de collisions triviales sur suites d'entiers ;
- distribution raisonnable sur un corpus de chaînes proches ;
- stabilité sur listes et dotted pairs ;
- stabilité sur handles d'entités CAD.

Choix des capacités premières

Motivation

La capacité du vecteur de slots doit de préférence être un **nombre premier**.

Cela simplifie :

- la réduction modulo ;
- certaines pathologies de distribution ;
- la robustesse générale du sondage linéaire.

Table de nombres premiers

Le projet doit embarquer une **table de nombres premiers croissants** couvrant les tailles prévues.

Cette table sert à :

- choisir la capacité initiale ;
- choisir la prochaine capacité lors d'un rehash.

La table doit couvrir au minimum :

- les petites tailles pratiques de démarrage ;
- les tailles intermédiaires utilisées par les benchmarks ;
- plusieurs ordres de grandeur au-delà, afin d'éviter qu'une v1 soit bloquée par un plafond artificiel.

Une progression indicative acceptable est :

- 5, 11, 23, 47, 97, 197, 397, 797, 1597, ... ;
- ou une suite de premiers proches d'une croissance x2.

La suite exacte sera figée avant l'implémentation, mais elle doit respecter :

- ordre strictement croissant ;
- absence de doublons ;
- densité suffisante pour ne pas surallouer exagérément.

Format de stockage de la table

Comme la recherche du prochain premier ne se fait pas dans le chemin critique des accès, plusieurs formats sont possibles.

La recommandation de la v1 est :

- stocker la table sous forme d'une structure **statique, triée et documentée** ;
- privilégier une représentation simple à maintenir ;
- autoriser une projection auxiliaire sous forme d'arbre de décision si cela simplifie la recherche dichotomique.

Concrètement, les options recevables sont :

- liste triée de premiers ;
- vecteur interne si un vecteur suffisamment pratique existe déjà ;
- arbre binaire statique construit à partir d'une liste triée.

Décision provisoire :

- la **source de vérité** sera une liste triée de nombres premiers ;
- l'implémentation pourra construire une vue binaire auxiliaire si cela aide la recherche ;
- aucune génération dynamique de nombres premiers n'est requise en v1.

Recherche du prochain premier

Étant donné une demande de capacité minimale n , il faut retourner :

- le plus petit premier $p \geq n$ présent dans la table.

Propriétés attendues :

- si n est inférieur ou égal au premier minimal, retourner ce premier ;
- si n tombe entre deux premiers, retourner le plus petit premier supérieur ;
- si n dépasse la table embarquée, l'implémentation doit signaler clairement que la table de capacités est insuffisante.

La recherche recommandée est une **recherche dichotomique**.

Même si la table source est une liste, la spécification veut que l'algorithme conceptuel soit celui-ci :

1. borne basse ;
2. borne haute ;
3. pivot ;
4. réduction de l'intervalle jusqu'à sélection du premier $\geq n$.

Le but n'est pas tant l'optimisation absolue que :

- la prédictibilité ;
- la simplicité de justification ;
- la possibilité de remplacer plus tard la représentation sans changer la sémantique.

Taille minimale initiale

La `v1` doit fixer une taille minimale non nulle pour éviter les cas pathologiques sur les très petites tables.

Recommandation :

- même si `size` vaut 0 ou n'est pas fourni, la capacité réelle initiale est au moins égale à un petit premier, par exemple 5 ou 11.

Le choix exact sera harmonisé avec les benchmarks de démarrage.

Recherche du prochain premier

Étant donné une taille voulue n , on cherche :

- le plus petit premier $p > n$.

Comme cette recherche n'est pas dans le chemin critique des accès, mais dans celui des allocations/rehash, la `v1` peut :

- stocker les premiers dans une liste triée ;
- ou les organiser dans un arbre binaire de décision ;

- ou utiliser une recherche dichotomique sur une séquence indexable si une telle structure est disponible.

L'intention actuelle du projet est :

- stocker la table sous une forme permettant une recherche **dichotomique** ;
- documenter clairement l'algorithme de sélection du prochain premier.

Rehash et croissance

Déclenchement

Le rehash est déclenché quand :

`count > floor(size * rehash-threshold)=`

La valeur par défaut recommandée est :

- `rehash-threshold = 0.5`.

Cette valeur est cohérente avec le choix d'un sondage linéaire :

- elle limite la longueur moyenne des séquences de collisions ;
- elle évite de dégrader rapidement les performances après quelques suppressions et réinsertions ;
- elle simplifie l'analyse des benchmarks.

Nouvelle capacité

La nouvelle capacité minimale est calculée à partir de :

- la taille actuelle ;
- le paramètre `rehash-size` ;
- puis arrondie au premier immédiatement supérieur ou égal dans la table des nombres premiers.

Les politiques acceptables sont :

- multiplicateur, par exemple 2.0 ;
- incrément explicite, si documenté.

La v1 peut se limiter à une politique multiplicative.

Décision recommandée pour la v1 :

- `rehash-size` est un multiplicateur réel ;
- sa valeur par défaut est 2.0 ;
- la capacité cible brute est :

`ceil(size * rehash-size)`

- la capacité réellement allouée est ensuite :

```
next-prime(ceil(size * rehash-size))
```

Exemple :

- taille courante 97 ;
- **rehash-size** = 2.0 ;
- cible brute 194 ;
- prochaine capacité 197.

Cette règle doit aussi respecter le fait que la nouvelle capacité est suffisante pour réinsérer toutes les entrées actives sans repasser immédiatement au-dessus du seuil.

Autrement dit, la capacité retenue doit aussi vérifier :

```
new-threshold > count
```

ou, de façon plus robuste :

```
floor(new-size * rehash-threshold) > count
```

Si la capacité calculée par le simple multiplicateur ne suffit pas, il faut continuer à avancer dans la table des premiers jusqu'à satisfaire cette contrainte.

Tombstones et rehash

La présence de nombreux slots **deleted** peut dégrader fortement les performances, même si **ah-count** reste modéré.

La v1 doit donc déjà poser le principe suivant :

- un rehash peut être déclenché non seulement par le nombre d'entrées actives, mais aussi par l'accumulation de tombstones.

La politique exacte peut rester simple au départ, par exemple :

- compter séparément **active-count** et **deleted-count** ;
- déclencher un rehash si **active-count** + **deleted-count** dépasse le seuil de charge ;
- ou déclencher un rehash si **deleted-count** dépasse une fraction documentée de la capacité.

La règle précise pourra être gelée au moment des benchmarks, mais le compteur de tombstones doit faire partie du modèle de données dès la conception.

Rehash sans croissance

Il faut distinguer :

- **rehash avec croissance** ;
- **rehash de nettoyage** à capacité identique.

Le second cas est utile quand :

- la table contient trop de tombstones ;
- mais le nombre d'entrées actives ne justifie pas encore une capacité plus grande.

La `v1` peut donc autoriser les deux opérations :

- réallouer à une nouvelle capacité plus grande ;
- ou réallouer à la même capacité pour compacter.

Réinsertion

Après allocation du nouveau vecteur :

- on parcourt tous les slots de l'ancienne table ;
- on ignore les slots `empty` et `deleted` ;
- on réinsère chaque entrée dans la nouvelle table à partir de son nouveau modulo.

Contraintes :

- les tombstones ne doivent jamais être recopiés ;
- le compteur d'entrées actives doit être recalculé proprement ;
- le compteur de tombstones doit être remis à 0 ;
- le rehash ne doit pas changer le contenu logique de la table.

Invariants de croissance à tester

Les tests devront vérifier :

- qu'une table croît avant saturation ;
- qu'après croissance toutes les entrées sont toujours accessibles ;
- que la nouvelle capacité est bien un nombre premier prévu ;
- que le seuil calculé est cohérent avec `rehash-threshold` ;
- qu'un rehash de nettoyage supprime bien les tombstones ;
- qu'aucune entrée n'est perdue ni dupliquée.

Tests d'égalité

La `v1` doit au minimum documenter le support de :

- `eq` ;
- `equal`.

Optionnellement :

- `eq1` ;
- `equalp` si une sémantique claire existe en AutoLISP.

Chaque test pris en charge doit préciser :

- comment comparer les clés ;

- quelle fonction de hachage lui est associée ;
- quelles contraintes sont garanties.

Benchmarks futurs

Les benchmarks devront comparer :

- qualité de distribution de **ah-sxhash** ;
- coût de **gethash** sans collision ;
- coût moyen avec collisions modérées ;
- coût d'insertion avant et après rehash ;
- impact du seuil de charge ;
- impact des tombstones après de nombreuses suppressions ;
- performance de plusieurs stratégies de mélange.

Des scénarios spécifiques devront couvrir :

- clés entières séquentielles ;
- chaînes proches ;
- symboles ;
- listes structurellement voisines ;
- handles d'entités CAD.

Décisions déjà prises

- la structure sera une **table ouverte** ;
- la résolution des collisions sera un **sondage linéaire circulaire** ;
- la table grandira avant d'être dense ;
- la capacité utilisera de préférence des **nombres premiers** ;
- **sxhash** est une exigence de premier ordre, pas un détail d'implémentation.

Questions ouvertes

- quels tests d'égalité supporter dès la v1 ;
- quel algorithme de mélange retenir pour **sxhash** ;
- comment représenter au mieux les marqueurs **empty** et **deleted** ;
- faut-il compacter ou rehasher après trop de tombstones ;
- quelle table de nombres premiers embarquer par défaut.

Étapes suivantes

1. Fixer précisément l'API publique **ah-**.
2. Choisir et documenter l'algorithme de mélange pour **ah-sxhash**.
3. Définir la table de nombres premiers.
4. Écrire les tests de sémantique avant l'implémentation.

5. Implémenter ensuite la structure interne et ses benchmarks.